

The background is a dark blue gradient with a complex, abstract digital pattern. It features several glowing, three-dimensional cubes and rectangular blocks, some of which are interconnected by thin, glowing lines. The overall aesthetic is futuristic and technological, with a focus on data and connectivity. The text 'Unit 2: Database Design using ER Model' is prominently displayed in the upper left quadrant in a white, sans-serif font. Below the title, the name 'Piyush Pant' is written in a smaller, white font. The bottom right corner of the image is decorated with several large, semi-transparent circles in various colors, including purple, blue, and green, adding a modern, artistic touch to the design.

Unit 2: Database Design using ER Model

Piyush Pant

ER Model

- The **Entity–Relationship (ER) model** is a **conceptual framework** for designing databases.
- It was introduced by **Peter Chen (1976)**.
- Purpose: To represent **real-world objects (entities)**, their **properties (attributes)**, and **connections (relationships)** in a structured way before implementation.
- Example: A *Student* entity with attributes like StudentID, Name, and Age, related to a *Course* entity.

ER Diagram

An **ER diagram** is the **graphical representation** of the ER model.

It uses **symbols** to show entities, attributes, and relationships:

-  **Rectangle** → Entity (e.g., Student, Course)
-  **Ellipse** → Attribute (e.g., Name, Age)
-  **Diamond** → Relationship (e.g., Enrolls, BelongsTo)
-  **Primary Key** → Underlined attribute (e.g., StudentID)
-  **Foreign Key** → Attribute linking entities (e.g., CourseID in Enrollment)
-  **Cardinality** → Defines relationship type (1:1, 1:N, M:N)

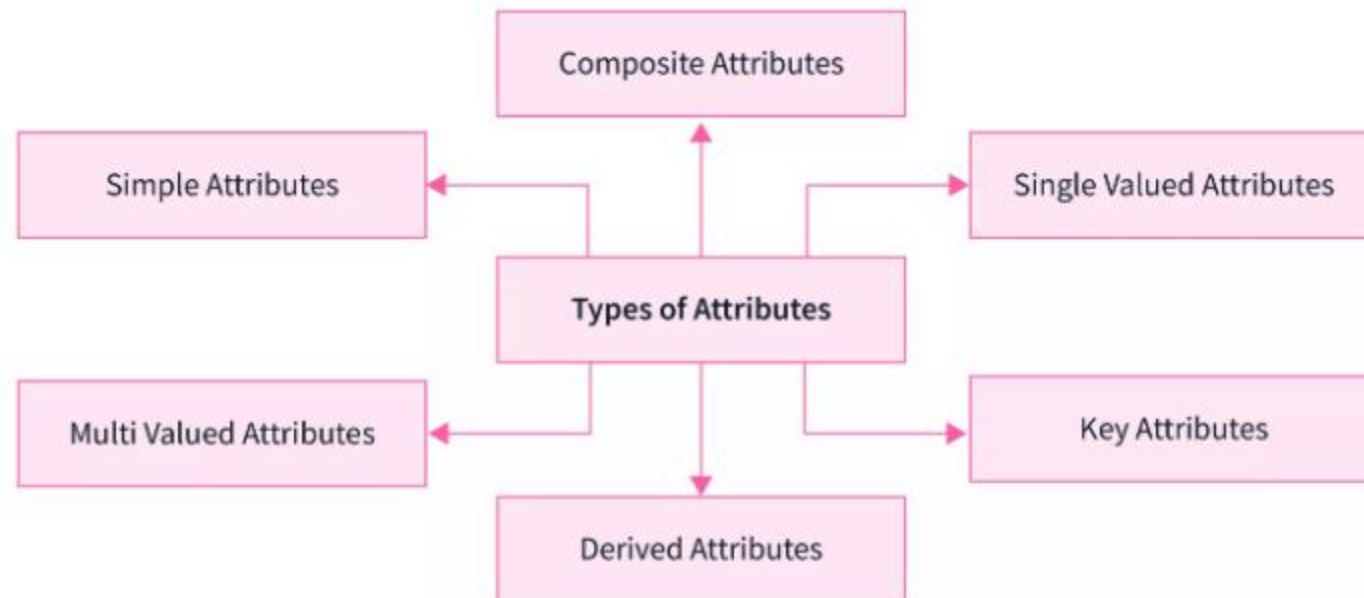
Attributes in ER Diagram

Attributes are the **properties of entities**. They describe details about each record in a table.

Types of Attributes:

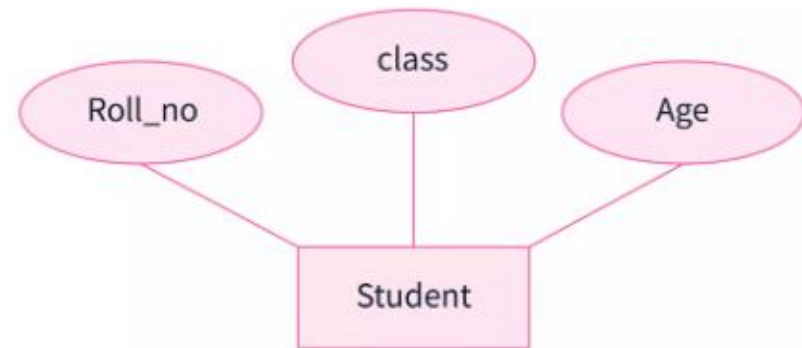
- **Simple Attribute** → Cannot be divided further (e.g., Age, RollNo).
- **Composite Attribute** → Can be split into subparts (e.g., Name → FirstName, LastName).
- **Derived Attribute** → Calculated from other attributes (e.g., Age derived from DateOfBirth).
- **Key Attribute** → Uniquely identifies an entity (e.g., StudentID).
- **Multivalued Attribute** → Can have multiple values (e.g., PhoneNumbers).

Types of Attributes in ER Diagram



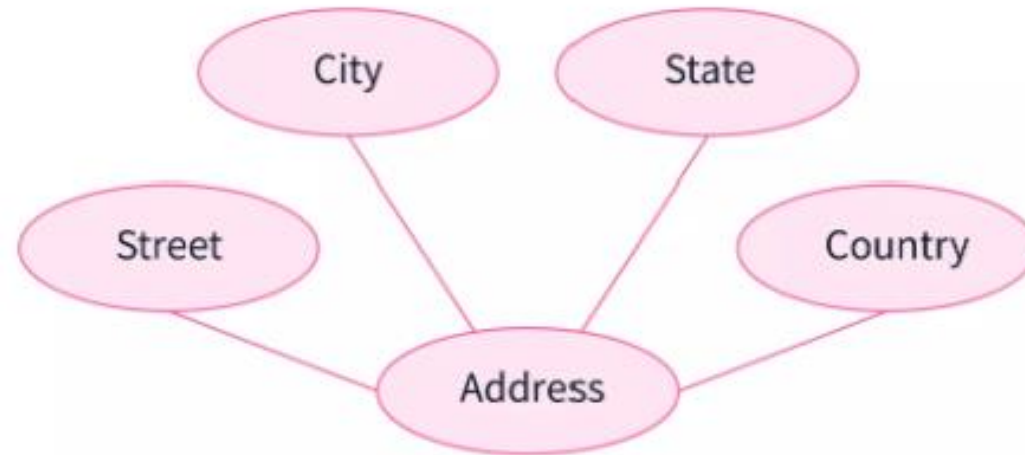
Simple Attributes

- Simple attributes in an ER model diagram are independent attributes that can't be classified further and also, can't be subdivided into any other component.
- These attributes are also known as **atomic attributes**.



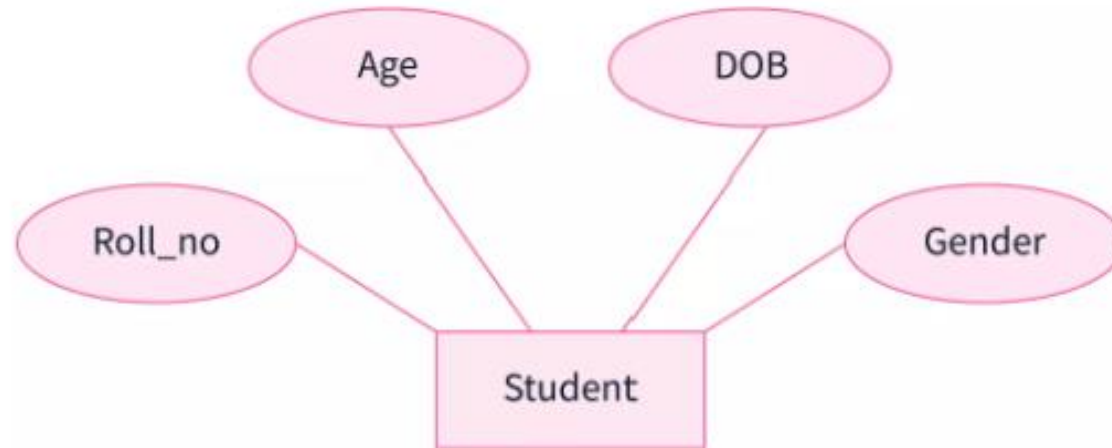
Composite Attributes

- Composite attributes have opposite functionality to simple attributes as we can further subdivide composite attributes into different components or sub-parts that form simple attributes.
- In simple terms, composite attributes are composed of one or more simple attributes.



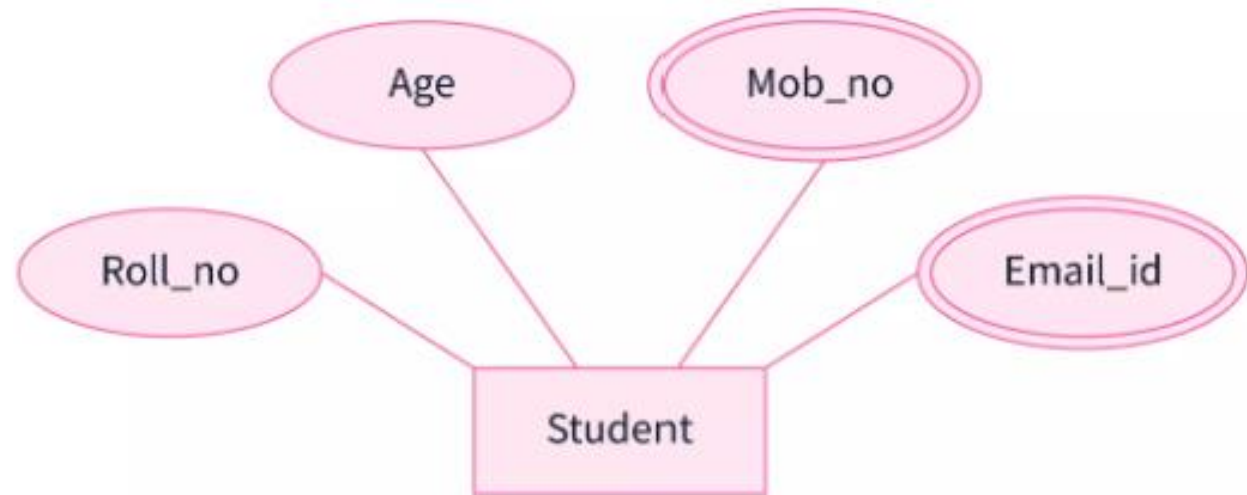
Single Valued Attribute

- Single-valued attributes are those attributes that consist of a single value for each entity instance and can't store more than one value.



Multi Valued Attribute

- Multi-valued attributes have opposite functionality to that of single-valued attributes,
- As the name suggests, multi-valued attributes can take up and store more than one value at a time for an entity instance from a set of possible values.
- These attributes are represented by **co-centric elliptical shape**

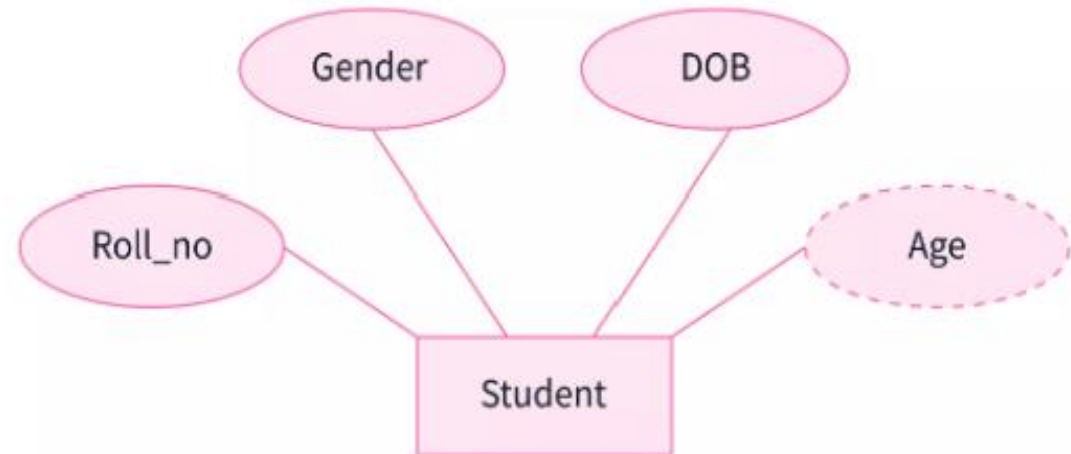


Derived Attributes

Derived attributes in DBMS are those attributes whose values can be derived from the values of other attributes. They are always dependent upon other attributes for their value.

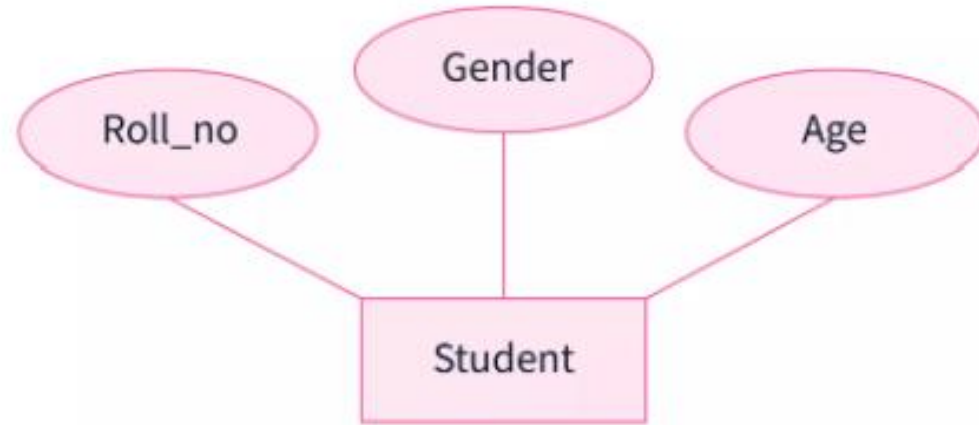
For example,

- DOB is a single-valued attribute and remains constant for an entity instance.
- From DOB, we can derive the Age attribute, which changes every year, and can easily calculate the age of a person from his/her date of birth value.
- Hence, the Age attribute here is derived attribute from the DOB single-valued attribute.



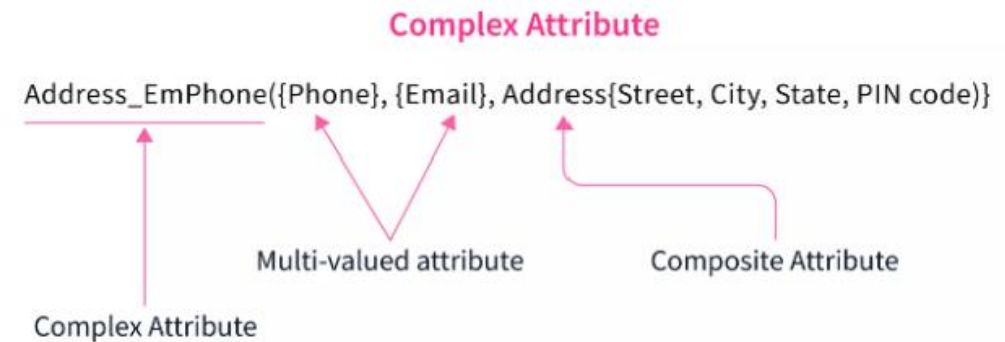
Key Attributes

- Key attributes are special types of attributes that act as the primary key for an entity and they can uniquely identify an entity from an entity set.
- The values that key attributes store must be unique and non-repeating.
- Eg: Roll_no



Complex Attributes

- Complex attributes are rarely used in DBMS.
- They are formed by the combination of multi-valued and composite attributes.
- These attributes always have many sub-sections in their values.



Stored Attributes

- Values of stored attributes remain constant and fixed for an entity instance and also, and they help in deriving the derived attributes.
- **For example**, the Age attribute can be derived from the Date of Birth attribute, and also, the Date of birth attribute has a fixed and constant value throughout the life of an entity. Hence, the Date of Birth attribute is a stored attribute.

